Formula Uno — Technical Deep Dive

Three questions answered, in full technical detail

1 • The fuel-load logic — the physics, and where 0.03 s/kg really comes from
2 • How this model compares to other public and academic F1 projects
3 • Exactly what is inside each model (spoiler: no neural networks)

A companion to the main documentation. Every number quoted here was produced by an actual run of the pipeline on real Bahrain data and verified reproducible before writing.

Question 1 — The Fuel-Load Logic

You asked two things: **how** fuel load affects lap time, and **how the figure of 0.03 seconds per kilogram was arrived at**. Both deserve a careful answer, and the second one comes with an important honesty note about how that number is actually used in the model.

A.1 Why extra mass makes a car slower — the physics

A heavier car is slower everywhere on the lap, through three distinct mechanisms:

1. Acceleration (Newton's second law)

Acceleration is force divided by mass: **a = F / m**. The engine delivers a roughly fixed driving force, so the more mass the car carries, the less it accelerates for that same force. On corner exits and along straights a fuel-heavy car simply picks up speed more slowly.

2. Braking (kinetic energy)

A moving car stores kinetic energy **KE = ½ m v²**, proportional to mass. Heavier means more energy the brakes must dissipate to stop, so the car needs to brake earlier (or harder) for each corner — costing time and shortening the following straight's useful length.

3. Cornering (lateral grip demand)

To hold a corner of radius r at speed v, the tyres must supply a lateral force of **m v² / r**. That required force grows with mass, but the grip a tyre can generate is limited — and, because of tyre *load sensitivity*, grip does not rise quite in proportion to the extra load. So a heavier car has to take the corner at a lower speed. Across a lap full of corners, this is usually the largest share of the fuel-weight penalty.

A.2 Where the 0.03 s/kg figure comes from

The number is an **established Formula 1 engineering rule of thumb**, not something I derived from first principles for this project. In the paddock it is usually stated as:

> **NOTE — The rule of thumb**
>
> "Ten kilograms of fuel is worth roughly three tenths of a second per lap." Ten kilograms → 0.30 seconds gives **0.03 seconds per kilogram per lap**. It is widely quoted by race engineers, technical directors and informed commentators for the current car generation.

Two important qualifications:

- **It is circuit-dependent.** The penalty is larger at tracks with many corners and heavy acceleration events (Monaco, Hungary — roughly 0.035–0.04 s/kg) and smaller at power circuits with long straights (Monza — roughly 0.02 s/kg). 0.03 is a sensible mid-range average.
- **It is easy to sanity-check.** A car starts a race with about 100 kg of fuel. At 0.03 s/kg that is around **3 seconds per lap** of pace locked up in fuel at the start, released gradually as the tank empties. That matches the well-known fact that cars lap several seconds quicker at the end of a race than at the start — good evidence the magnitude is right and worth modelling.

A.3 The crucial clarification: 0.03 is NOT hard-coded into the model

This is the part most people get wrong when they hear the number, so it is worth being precise. The **FuelLoad** feature the model reads is expressed in **kilograms**, computed purely from lap number and race length:

```
FuelLoad = 105.0 - (LapNumber - 1) * (105.0 / total_laps)      # kilograms remaining
```

The constant 0.03 **appears nowhere in the code**. It served two jobs only: it motivated adding a fuel feature at all, and it let me sanity-check that the effect is large enough to bother modelling. The model is then handed the fuel mass in kilograms and **learns the seconds-per-kilogram relationship itself**, directly from tens of thousands of real laps.

> **VERIFIED — Why this matters**
>
> Because the coefficient is learned rather than assumed, the model is never bound by my rule of thumb. Through its decision-tree splits XGBoost can effectively fit a *different* fuel sensitivity for different circuits, compounds and eras — discovering the real relationship in the data rather than parroting a number I typed in. The 0.03 figure is scaffolding for human understanding, not a parameter of the model.

A.4 An honest nuance — and a correction to something I said earlier

When I first proposed the fuel feature I called it “the biggest single win.” Having looked more carefully, I want to be precise rather than oversell it.

Within a **single race**, FuelLoad is a perfectly straight-line function of LapNumber — it is just *105 minus a constant times lap number*. Mathematically the two carry the same information, so for a single-circuit model a decision tree can split on either one and FuelLoad adds little that LapNumber did not already provide.

Its real value shows up **across circuits of different lengths**. Consider lap 20 at two tracks:

On lap 20	Race length	Raw LapNumber sees	FuelLoad sees
Bahrain	57 laps	20	70.0 kg remaining
Spa	44 laps	20	59.7 kg remaining

Raw lap number calls both situations “20” and treats them alike. FuelLoad correctly says the Spa car is 10 kg lighter and therefore in a different physical state. So the feature re-expresses race progress on a common physical scale, letting a **combined multi-circuit model transfer fuel and tyre knowledge between tracks of different lengths**. That is its genuine contribution.

> **HONEST CAVEAT — The precise claim**
>
> Fuel load’s benefit is largest in the **multi-circuit** model. For the single-circuit Bahrain model, the larger measured accuracy gains actually came from the squared tyre-life term, filtering out Safety-Car laps, the hyperparameter tuning, the dedicated qualifying model — and, above all, fixing the bug where “round 1” loaded the wrong circuit entirely. I would rather give you the accurate picture than the tidy headline.

Question 2 — How Do We Compare to Similar Projects?

To judge whether the model is severely off or in an acceptable range, I gathered reported error figures from public and academic F1 lap-time projects and lined them up against ours. First, though, three caveats, because a naive number-to-number comparison would be misleading.

B.1 Why exact cross-project comparison is imperfect

- **Different data scales.** Several public projects *normalise* lap times to a unitless scale before modelling. Their impressive-looking error of “0.04” is in normalised units, not seconds — it cannot be compared directly to our seconds.
- **Different validation protocols.** Most public projects use a *random* train/test split, in which laps from the very race being tested can leak into the training set — an easier task. Ours uses a *temporal cross-year* split: train on 2022–2024, test on an entirely unseen 2025. The same model scores worse under our stricter protocol, so our numbers are conservative by comparison.
- **Different metrics.** We report RMSE, which punishes large misses more heavily than MAE. Since RMSE is always at least as large as MAE, comparing our RMSE against another project’s MAE quietly handicaps us.

B.2 The comparison

With those caveats in mind, here is where the numbers land. Our own figures (highlighted) come from a verified run on real Bahrain data.

Source / project	Approach	Reported error	Validation
This project — 2024 Bahrain (in-sample)	XGBoost	Fleet RMSE 0.162 s	in-sample
This project — 2025 Bahrain (out-of-sample)	XGBoost	Fleet RMSE 1.492 s	cross-year (hard)
This project — 2025 qualifying	XGBoost (dedicated)	RMSE 1.009 s	out-of-sample
Academic surveys (typical)	Random Forest	MAE 0.5 – 1.2 s (R2 0.80–0.90)	per track/driver
Babbar — 6-model bake-off	RF / XGBoost best; NN worst	MSE ~0.0018 (normalised)	random split
Pusuluri — qualifying study	Random Forest	aggregate MAE ~8 – 12 s (preliminary)	random split

B.3 The verdict

- **In-sample (0.162 s)** is excellent — comfortably inside, and in fact below, the best reported academic band (their 0.5–1.2 s is MAE; ours is the stricter RMSE, yet still smaller).
- **Out-of-sample (1.492 s)** is squarely in the acceptable-to-good range — *not* severely off — and it is achieved under a materially harder validation protocol than the projects it is being compared with. Almost all of the gap above the in-sample figure is the handful of 2025 rookies and team-changing drivers the model could not possibly have learned (covered in the main doc).
- **Qualifying (1.009 s)** beats the comparable public qualifying project’s aggregate error by a wide margin and approaches the best specialised results.

> **VERIFIED — Independent validation of the model choice**
>
> The one study that put six algorithms head-to-head on F1 lap times found the **tree ensembles (Random Forest, XGBoost, LightGBM) best and the neural network worst**. That is exactly the design this project settled on — strong outside evidence that reaching for XGBoost rather than a neural network was the right call for this data.

Bottom line: the model is not severely off. It sits in the acceptable-to-strong range, and arguably stronger than face value, because it is validated more strictly than most projects it is measured against.

Question 3 — What Is Actually Inside Each Model?

KEY POINT — The headline, stated plainly: there are no neural networks

Your question mentioned "neural networks and everything," so the single most important thing to know is this: **this project uses zero neural networks — no deep learning of any kind.** Both the race-pace model and the qualifying model are **XGBoost gradient-boosted decision trees.** If an interviewer asks "did you use deep learning?", the correct, confident answer is: "No — gradient-boosted trees, and deliberately so, for reasons I can justify." The rest of this section arms you with those reasons.

C.1 Building the idea up, so you can explain it cold

XGBoost is best understood as the top of a short ladder of ideas. Each rung fixes a weakness of the one below.

- **Decision tree.** A flowchart of yes/no questions on the features — "tyre life above 15 laps? → is the compound hard? → …" — ending in a predicted lap time. A single tree is intuitive but weak: on its own it either oversimplifies or memorises noise.
- **Ensemble.** Combine many trees and average them. A crowd of weak predictors reliably beats any single one — errors cancel out.
- **Bagging vs boosting.** A Random Forest builds many *independent* trees and averages them (bagging). **Boosting** instead builds trees *in sequence*, each one trained to correct the errors the previous trees still make.
- **Gradient boosting.** Each new tree is fitted to the *residual error* of the ensemble so far — literally "where are we still wrong, and by how much?" — and nudges the prediction toward the truth. Do this hundreds of times and the ensemble becomes very accurate.
- **XGBoost.** A fast, heavily optimised implementation of gradient boosting with built-in *regularisation* (penalties that discourage overfitting) and row/column subsampling. It is the de-facto industry standard for tabular data and repeatedly wins data-science competitions on this kind of problem.

C.2 The two models: identical machinery, different diet

The system trains **two** XGBoost models. They use the exact same algorithm and the same ten features; they differ only in the data they are fed and the job they do.

	Race-pace model	Qualifying model
Algorithm	XGBoost (gradient-boosted trees)	XGBoost (gradient-boosted trees)
Features	The same 10 engineered features	The same 10 engineered features
Training data	~2,831 race laps (2022–24)	~253 qualifying laps (2022–24)
Target	Race lap time (seconds)	Qualifying lap time (seconds)
Fuel assumption	~105 kg, burning down	Fixed light load (~5 kg)
Job	Reconstruct the full race	Predict the grid order

The qualifying model's much smaller training set (only a few flying laps per driver per weekend) is a real limitation — its cross-validation error is high and variable (about 4.4 s, give or take 2.2). Yet on the 2025 test it predicted the pole time to within about a tenth of a second, because single-lap pace at a given circuit is quite stable year to year.

C.3 Every hyperparameter, explained

These are the settings that control how the trees are built. Our tuned values:

Setting	Value	What it controls
n_estimators	2000	Number of trees / boosting rounds — more trees, finer fit
learning_rate	0.02	How much each tree contributes; small steps reach a better optimum
max_depth	6	How many questions deep each tree can go — its complexity
min_child_weight	5	Minimum real laps behind any leaf — blocks overfitting to noise
subsample	0.8	Each tree sees a random 80% of the laps — adds robustness
colsample_bytree	0.8	Each tree sees a random 80% of the features
random_state	42	Fixes randomness so runs are reproducible
n_jobs	-1	Use all CPU cores

> **NOTE — The learning-rate / tree-count relationship**
>
> n_estimators and learning_rate work together. A small learning rate (0.02) means each tree only nudges the prediction a little, so you need many trees (2000) to arrive — but the many small, careful steps reach a better final answer than a few large ones. It is the single most important tuning trade-off in gradient boosting.

C.4 Why XGBoost and NOT a neural network

This is the technical justification to have ready. Every point below is a reason trees beat deep learning *for this particular problem*:

- **Dataset size.** We have about 2,831 race laps and only 253 qualifying laps. Neural networks are data-hungry and typically need tens of thousands to millions of examples; on a set this small they overfit badly. Tree ensembles are at their best on small-to-medium tabular data.
- **Data type.** The data is tabular — rows of mixed categorical and numeric features — with no image, audio or long-sequence structure for a convolutional or recurrent network to exploit. Trees handle mixed feature types natively.
- **Feature interactions come for free.** A tree split chain automatically captures interactions like driver × compound × tyre age, without any manual feature crossing.
- **Interpretability.** XGBoost reports feature-importance (gain) scores, telling you *which* inputs drive the prediction. For a project you must be able to explain, that transparency is invaluable — a neural network is a black box by comparison.
- **Practicality.** No GPU needed, training is seconds not hours, and trees are insensitive to feature scaling (no normalisation step to get wrong).
- **Empirical evidence.** As noted in Question 2, the one head-to-head study on this exact problem found a neural network to be the *worst* of six models, with tree ensembles best.

C.5 The training procedure, end to end

For each of the two models the same disciplined sequence runs:

- Assemble the feature table — ten feature columns plus the lap-time target.

- Run **5-fold cross-validation**: split the laps into five parts, train on four and test on the fifth, rotate through all five, and average the error. This “CV RMSE” is an honest estimate of performance on unseen data, computed before any race is simulated (race model ~0.79 s; qualifying model ~4.4 s, its high value flagging the small-data limitation).
- Fit the final model on all the laps.
- Report training MAE and RMSE. A low training error beside a higher CV error is the standard overfitting check — the gap here is modest and expected.
- Save (pickle) the trained model to disk so it can be reused without retraining.

C.6 Where machine learning stops and plain code begins

An honest technical account must draw this line clearly, because it is exactly what a sharp interviewer will probe:

> **NOTE — The learned part is small and well-contained**
>
> The **only** machine-learning component is the pair of XGBoost models, and all they ever do is predict **one lap time at a time**. Everything else in the system is ordinary deterministic code: accumulating those lap times, sorting cars by total time to get positions, injecting real historical pit in/out-lap times, resetting tyre age at a stop. The race engine is a hand-written simulator, *not* a learned model.

So the accurate one-sentence description of the architecture is: **a learned pace oracle wrapped in a deterministic race simulator.** Being able to say precisely which parts are learned and which are not is a mark of understanding the system rather than just running it.

C.7 The recap you can memorise

> **VERIFIED — One paragraph that covers it**
>
> “Two XGBoost gradient-boosted-tree regressors — one for race pace, one for qualifying — each reading ten engineered features (driver, team, circuit, year-in-era, lap number, tyre compound, tyre age, tyre age squared, fuel load and grid position) to predict a single lap time in seconds. They are wrapped in a deterministic lap-by-lap simulator that accumulates the predicted times and lets finishing order emerge. No neural networks: gradient-boosted trees were chosen because the dataset is small, tabular, and needs to stay interpretable.”

Sources

Comparison figures and the model-bake-off evidence in Question 2 were drawn from the following public and academic F1 lap-time projects:

- Babbar, T. — “Predicting F1 Lap Times: A Comparison of ML Models” (six-model bake-off; neural network worst, tree ensembles best): medium.com/@tapanbabbar
- Pusuluri, S. — “Who’s Fastest? Predicting F1 Qualifying Lap Times using Machine Learning”: medium.com/@SaiUbc
- vicmcl — “ml-laptime” (FastF1 + XGBoost lap-time modelling): github.com/vicmcl/ml-laptime
- “Predicting Lap Times of Formula 1 Races using ANN” — IJSREM (neural-network approach): ijsrem.com
- “Deep Neural Network-based lap time forecasting of Formula 1 Racing” — ResearchGate.
- “A State-Space Approach to Modeling Tire Degradation in Formula 1 Racing” — arXiv:2512.00640.

Every figure attributed to “this project” was produced by a verified run on real Bahrain data (2022–2024 train, 2025 out-of-sample test) and reproduced before writing.